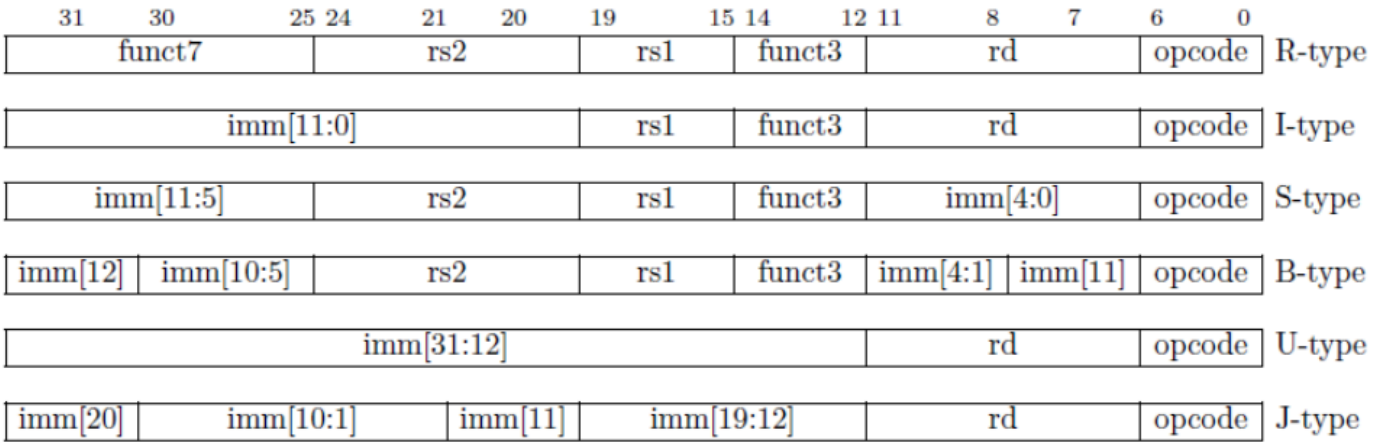
x86

Поле	Размер	Назначение
OPCODE	6	Тип операции
D	1	Направление данных
W	1	Размер операнда
MOD	2	Тип адресации
REG	3	Регистр-операнд
R/M	3	Второй операнд или способ адресации

RISC-V

Основные поля инструкции (например, в формате R-типа):

- `opcode` (7 бит): Определяет тип инструкции.
- `rd` (5 бит): Регистр, в который будет записан результат.
- `funct3` (3 бита): Определяет конкретную операцию внутри семейства инструкций.
- `rs1` (5 бит): Первый регистр-источник.
- `rs2` (5 бит): Второй регистр-источник.
- `funct7` (7 бит): Определяет дополнительную функцию для некоторых инструкций.


31 30 25 24 21 20 19 15 14 12 11 8 7 6 0
funct7 rs2 rs1 funct3 rd opcode R-type
imm[11:0] rs1 funct3 rd opcode I-type
imm[11:5] rs2 rs1 funct3 imm[4:0] opcode S-type
imm[12] imm[10:5] rs2 rs1 funct3 imm[4:1] imm[11] opcode B-type
imm[31:12] rd opcode U-type
imm[20] imm[10:1] imm[11] imm[19:12] rd opcode J-type

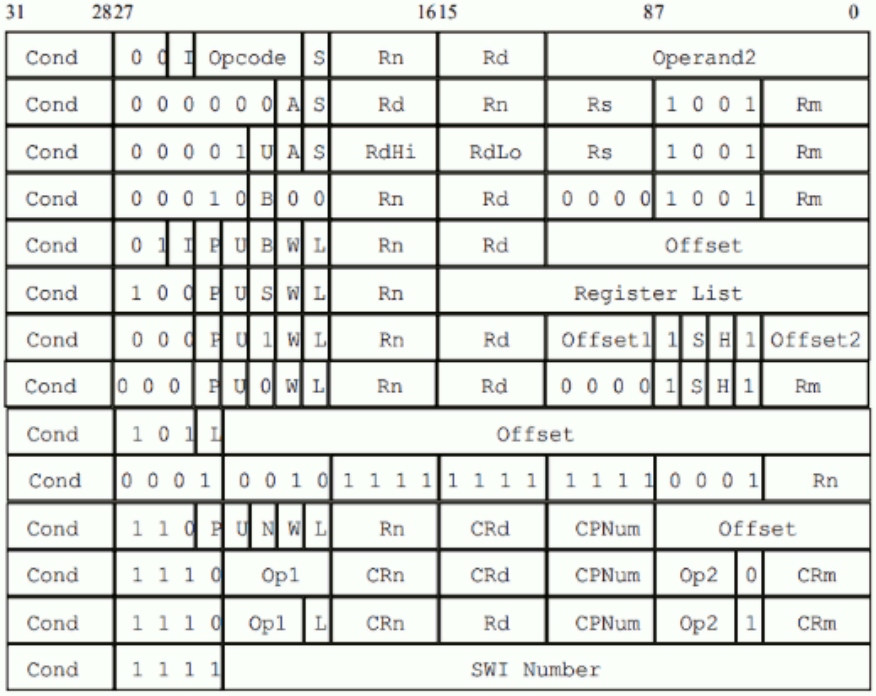
RISC-V расширение C

Format	Meaning	15 14 13	12	11 10	9 8 7	6 5	4 3 2	1 0
CR	Register	funct4		rd/rs1		rs2		op
CI	Immediate	funct3	imm	rd/rs1		imm		op
CSS	Stack-relative Store	funct3	imm			rs2		op
CIW	Wide Immediate	funct3	imm				rd′	op
CL	Load	funct3	imm		rs1′	imm	rd′	op
CS	Store	funct3	imm		rs1′	imm	rs2′	op
CB	Branch	funct3	offset		rs1′	offset		op
CJ	Jump	funct3	jump target					op

Table 1.1: Compressed 16-bit RVC instruction formats.

ARM

ARM Instruction Set Format



Instruction type

- Data processing / PSR Transfer
- Multiply
- Long Multiply **(v3M / v4 only)**
- Swap
- Load/Store Byte/Word
- Load/Store Multiple
- Halfword transfer : Immediate offset **(v4 only)**
- Halfword transfer: Register offset **(v4 only)**
- Branch
- Branch Exchange **(v4T only)**
- Coprocessor data transfer
- Coprocessor data operation
- Coprocessor register transfer
- Software interrupt

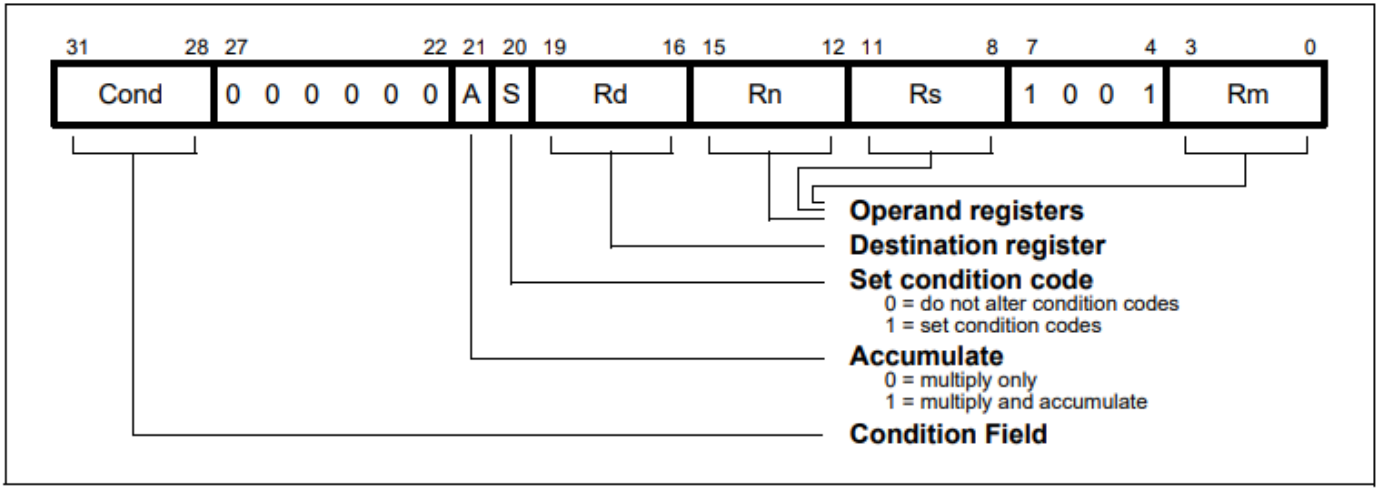


Figure 4-12: Multiply instructions

Rd:=Rm*Rs.

Rn is ignored, and should be set to zero for compatibility with possible future upgrades to the instruction set.

Multiply-accumulate form:

Rd:=Rm*Rs+Rn,
which can save an explicit ADD instruction in some circumstances.

Both forms of the instruction work on operands which may be considered as **signed** (2's complement) ***or*** **unsigned** integers.

The results of a signed multiply and of an unsigned multiply of 32 bit operands differ only in the upper 32 bits - *the low 32 bits results are identical.*

As these instructions only produce the low 32 bits of a multiply, they can be used for both signed and unsigned multiplies. For example consider the multiplication of the operands:

Operand A	Operand B	Result
0xFFFFFFF6	0x00010100	0xFFFFFF38

If the operands are interpreted as signed

- Operand A has the value -10,
- operand B has the value 20,
- the result is -200 which is correctly represented as 0xFFFFFF38

Operand A	Operand B	Result
0xFFFFFFF6	0x00010100	0xFFFFFF38

If the operands are interpreted as unsigned

- Operand A has the value 4294967286,
- operand B has the value 20
- the result is 85899345720, which is represented as 0x13FFFFFF38, so the least significant 32 bits are 0xFFFFFF38.

Current Program Status Register flags

is optional and is controlled by the S bit in the instruction.
The N (Negative) and Z (Zero) flags are set correctly on the result (N is made equal to bit 31 of the result, and Z is set if and only if the result is zero).
The C (Carry) flag is set to a meaningless value and the V (oVerflow) flag is unaffected.

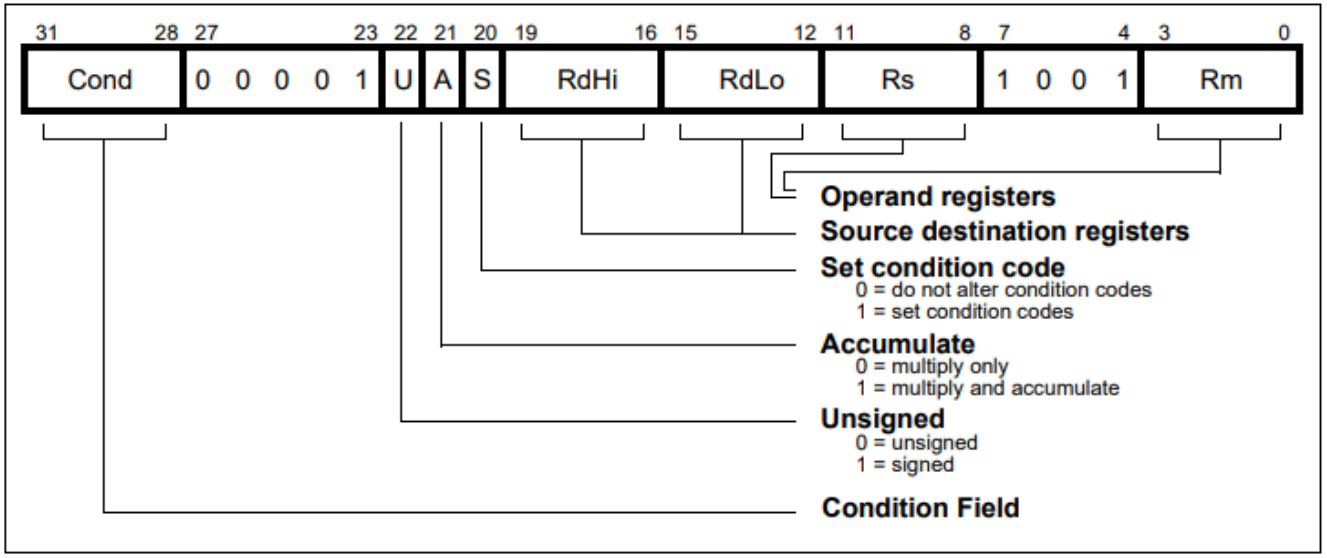


Figure 4-13: Multiply long instructions

The multiply forms (UMULL and SMULL)

- take two 32 bit numbers and multiply them to produce a 64 bit result of the form
- RdHi,RdLo := Rm * Rs.
- The lower 32 bits of the 64 bit result are written to RdLo,
- the upper 32 bits of the result are written to RdHi.

The multiply-accumulate forms (UMLAL and SMLAL)

- take two 32 bit numbers, multiply them and add a 64 bit number to produce a 64 bit result of the form
- RdHi,RdLo := Rm * Rs + RdHi,RdLo.
- The lower 32 bits of the 64 bit number to add is read from RdLo.
- The upper 32 bits of the 64 bit number to add is read from RdHi.
- The lower 32 bits of the 64 bit result are written to RdLo.
- The upper 32 bits of the 64 bit result are written to RdHi.
- The UMULL and UMLAL instructions treat all of their operands as unsigned binary numbers and write an unsigned 64 bit result.
- The SMULL and SMLAL instructions treat all of their operands as two’s-complement signed numbers and write a two’scomplement signed 64 bit result.

Motorola

15 - 12	11 - 9	8 - 6	5 - 3	2	1 - 0
OPCODE	REGISTER1	MODE	REGISTER2	SIZE	CONDITION

Инструкция ADD

15-12	11-9	8-6	5-3	2	1-0
1101	010	000	001	0	00